

ERP VENTAS · SPRING BOOT 4 · JAVA 21

Anotaciones de Spring Boot y funcionamiento de JWT

Referencia completa de todas las anotaciones usadas en **Ventas-Backend**, con ejemplos reales del código, más una explicación a fondo de cómo funciona la autenticación con JSON Web Tokens.

Proyecto: **Ventas-Backend** — API REST de punto de venta

Documento complementario de la *Guía de Spring Boot*

Estado del código: 12 controllers · ~56 endpoints · 34 permisos · 130 tests en verde

Índice

1. Qué es una anotación y cómo las procesa Spring
 - Meta-anotaciones, reflexión y proxies
2. Mapa general de anotaciones del proyecto
3. Anotaciones de arranque y del contenedor
4. Anotaciones web (controllers REST)
5. Inyección de dependencias y configuración
6. Anotaciones de seguridad y permisos
7. Anotaciones de persistencia (JPA/Hibernate)
8. Transacciones, consultas y fetching
9. Manejo de excepciones
10. Validación de datos (Bean Validation)
11. Serialización JSON (Jackson) y Lombok
12. Anotaciones de testing
13. Tabla maestra: anotación → paquete → uso
14. JWT explicado a fondo
 - Qué es y por qué se usa
 - Estructura: header.payload.signature
 - Base64Url y firma HS256
 - Claims y expiración
 - Flujo completo en este proyecto
 - Implementación real (JwtUtil, JwtFilter, SecurityConfig, mapper)
 - Seguridad, riesgos y buenas prácticas
15. Preguntas de entrevista (anotaciones + JWT)
16. Glosario

1. Qué es una anotación y cómo las procesa Spring

Una **anotación** en Java es un metadato que se coloca sobre clases, métodos, campos o parámetros (por ejemplo `@Service`). Por sí sola **no hace nada**: es información que otro programa lee y decide en consecuencia. Ese "otro programa" es Spring, y lo hace en tres momentos:

1.1 Escaneo del classpath (arranque)

Al arrancar, Spring recorre los paquetes bajo la clase `@SpringBootApplication` buscando clases anotadas con estereotipos (`@Component`, `@Service`, `@Repository`, `@RestController`, `@Configuration`). Por cada una crea un **bean**: un objeto que vive dentro del contenedor y que Spring administra por ti.

1.2 Inyección de dependencias

Cuando Spring construye un bean, mira su constructor o sus campos y **inyecta** los beans que necesita. En este proyecto se usa casi siempre **inyección por constructor** (más segura: los campos quedan `final` y no hay ciclos silenciosos). Ejemplo real en `SaleController`:

```
@RestController
@RequestMapping("/api/sales")
public class SaleController {
    private final SaleService service;

    // Spring ve este único constructor y pasa el bean SaleService
    public SaleController(SaleService service){
        this.service = service;
    }
}
```

1.3 Meta-anotaciones y proxies

Muchas anotaciones de Spring son **meta-anotaciones**: `@RestController` es en realidad `@Controller` + `@ResponseBody`; `@Service` es `@Component`. Otras activan **proxies**: al anotar un método con `@Transactional` o `@PreAuthorize`, Spring envuelve el objeto en un proxy que ejecuta código extra antes/después de la llamada (abrir/cerrar transacción, verificar permisos). Por eso **estas anotaciones deben ir en métodos públicos y llamarse desde fuera del bean** (no funcionan en llamadas internas del propio objeto).

Idea clave para la entrevista

"Las anotaciones son metadatos. Spring las descubre con classpath scanning, las usa para crear el ApplicationContext, construye proxies AOP para las que lo requieren (`@Transactional`, `@PreAuthorize`) e inyecta dependencias por constructor."

2. Mapa general de anotaciones del proyecto

Estas son **todas** las anotaciones que aparecen en el código de `Ventas-Backend` (excluyendo las etiquetas de Javadoc como `@param` y direcciones de correo). La última columna indica un ejemplo real.

Grupo	Anotaciones	Dónde se usan
Arranque / contenedor	@SpringBootApplication, @Configuration, @Bean, @Component, @Service, @Repository, @RestController, @RestControllerAdvice, @Order	VentasApplication, SecurityConfig, bootstraps, services, repos, controllers
Web / REST	@RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping, @PathVariable, @RequestParam, @RequestBody	los 12 controllers
Inyección / config	@Autowired, @Value, @ConfigurationProperties, @ConditionalOnProperty, @EnableScheduling, @Scheduled	JwtUtil, TerminalConfig, PaymentMonitorJob, terminal impls
Seguridad	@EnableMethodSecurity, @PreAuthorize	SecurityConfig y todos los controllers
Persistencia (JPA)	@Entity, @Table, @Id, @GeneratedValue, @Column, @UniqueConstraint, @OneToOne, @OneToMany, @ManyToOne, @ManyToMany, @JoinColumn, @JoinTable, @Enumerated, @PrePersist, @PreUpdate	12 entidades en model/entity
Transacciones / queries	@Transactional, @Query, @EntityGraph	service/impl, repositories, bootstraps
Excepciones	@RestControllerAdvice, @ExceptionHandler	GlobalExceptionHandler
Validación	@Valid, @NotBlank, @Min, @Size	Disponible pero hoy sin uso: la dependencia está en el pom y el handler está listo; las reglas se validan a mano en los service/impl
JSON (Jackson)	@JsonIgnore	RoleEntity, PermissionEntity, CategoryEntity
Lombok	@Data, @Builder, @NoArgsConstructor, @AllArgsConstructor, @RequiredArgsConstructor, @Slf4j	entidades, DTOs, GlobalExceptionHandler, TestCardRepository
Testing	@SpringBootTest, @DataJpaTest, @WebMvcTest, @ExtendWith, @WithMockUser, @Mock, @MockitoBean, @InjectMocks, @BeforeEach, @Test, @Import, @TempDir	las 21 clases de src/test

3. Anotaciones de arranque y del contenedor

@SpringBootApplication

Combina `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`. Es el punto de entrada: habilita la autoconfiguración (Spring mira el classpath y configura solo lo que detecta) y escanea el paquete base y sus subpaquetes.

```
@SpringBootApplication
@EnableScheduling // habilita @Scheduled (PaymentMonitorJob)
public class VentasApplication {
    public static void main(String[] args) {
        SpringApplication.run(VentasApplication.class, args);
    }
}
```

@Configuration y @Bean

`@Configuration` marca una clase que define beans manualmente. `@Bean` registra **un objeto concreto** en el contenedor cuando no puedes (o no quieres) anotar la clase (típicamente clases de librerías). En este proyecto `SecurityConfig` crea el `PasswordEncoder`, el `CorsConfigurationSource` y la `SecurityFilterChain`.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // hashing de contraseñas
    }
}
```

@Configuration vs @Component: ambos son beans, pero `@Configuration` indica a Spring que las llamadas entre métodos `@Bean` deben devolver siempre el mismo objeto (singleton), gracias a un proxy de la clase.

@Component, @Service, @Repository, @RestController

Son **estereotipos**: todos registran la clase como bean. Se diferencian por intención y por comportamientos extra:

Anotación	Intención	Extra
@Component	Bean genérico	—
@Service	Lógica de negocio	Semántica
@Repository	Acceso a datos	Traduce excepciones de persistencia a <code>DataAccessException</code>
@RestController	API REST	@Controller + @ResponseBody (serializa lo que devuelva el método a JSON)

```
@Service
public class AuthImpl implements AuthService { ... } // negocio

@Repository
public class TestCardRepository { ... } // datos (en memoria)

@RestController
@RequestMapping("/api/sales")
public class SaleController { ... } // HTTP
```

@RestControllerAdvice

Igual que `@ControllerAdvice` (intercepta todos los controllers) pero además serializa la respuesta a JSON. Se usa una sola vez, en `GlobalExceptionHandler`, para centralizar el formato de error.

@Order

Define el orden de ejecución entre beans del mismo tipo. Aquí se aplica a los `CommandLineRunner` de bootstrap, que deben sembrar en secuencia: `RoleBootstrap` → `AdminBootstrap` → `PermissionBootstrap` → `RolePermissionBootstrap`. Menor valor = se ejecuta antes.

4. Anotaciones web (controllers REST)

@RequestMapping y verbos HTTP

`@RequestMapping("/api/sales")` fija la ruta base de la clase. Cada método se mapea a un verbo HTTP con `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` o `@PatchMapping`. Ejemplo real, `SaleController`:

```
@RestController
@RequestMapping("/api/sales")
public class SaleController {
    private final SaleService service;

    public SaleController(SaleService service){ this.service = service; }

    @PreAuthorize("hasAuthority('CREAR_VENTAS')")
    @PostMapping
    public ResponseEntity<SaleResponse> processSale(@RequestBody SaleRequest request){
        return ResponseEntity.ok(service.processSale(request));
    }

    @PreAuthorize("hasAuthority('VER_VENTAS')")
    @GetMapping("/{id}")
    public SaleDetailHistoryResponse getSaleById(@PathVariable Long id){
        return service.getSaleById(id);
    }
}
```

@PathVariable, @RequestParam y @RequestBody

Anotación	De dónde lo toma	Ejemplo
@PathVariable	Parte variable de la URL	GET /api/sales/42 → @PathVariable Long id
@RequestParam	Query string (?q=cafe&top=10)	GET /api/products/search?q=cafe
@RequestBody	Cuerpo JSON de la petición	POST /api/sales con { ... }

```
@GetMapping("/search")
public List<ProductDto> search(@RequestParam String q){ ... }

@GetMapping("/barcode/{bc}")
public ProductDto byBarcode(@PathVariable String bc){ ... }
```

@CrossOrigin (histórico)

Permitía CORS a nivel de controller. En este proyecto **ya no se usa**: el CORS se configuró de forma global en `SecurityConfig` con la propiedad `#{CORS_ALLOWED_ORIGINS}`. Los comentarios de `ProductController` y `SaleController` lo dejan explícito.

5. Inyección de dependencias y configuración

@Autowired

Pide que Spring inyecte una dependencia. Es **implícito** cuando hay un solo constructor, así que en el código de producción casi no aparece. Sí se usa en tests (campos `MockMvc`) y en algún runner. La recomendación es **inyección por constructor**.

```
@Autowired
private MockMvc mockMvc; // en los tests @WebMvcTest
```

@Value

Inyecta un valor de configuración (de `application.yaml`, variable de entorno o `application-local.yaml`). Admite un valor por defecto tras dos puntos.

```
@Value("${JWT_SECRET}")      private String secretKey; // obligatorio
@Value("${app.frontend-url}") private String frontendUrl;
@Value("${CORS_ALLOWED_ORIGINS:http://localhost:4200}") private String allowedOrigins;
```

Seguridad

Si una variable aparece como `${VAR}`

sin default

, la app no arranca si falta. Por eso localmente existe `application-local.yaml` (ignorado por Git) y en Railway los valores van en las Variables del servicio. Un secreto filtrado a Git permitiría

forjar tokens de ADMIN

.

@ConfigurationProperties

Enlaza un bloque entero del YAML a los campos de un objeto (tipado y agrupado). Es más limpio que muchos `@Value`. Se usa en `TerminalConfig`:

```
@Data
@Component
@ConfigurationProperties(prefix = "payment.terminal")
public class TerminalConfig {
    private String type;      // payment.terminal.type
    private String host;
    private int port;
    private String merchantId;
    private boolean sslEnabled;
    // ...
}
```

@ConditionalOnProperty

Registra un bean **solo si** una propiedad tiene cierto valor. Es la pieza que permite cambiar de terminal de pagos sin tocar código: si `payment.terminal.type=SIMULATED` se activa la simulada; si es `PHYSICAL`, la real.

```
@ConditionalOnProperty(name = "payment.terminal.type", havingValue = "SIMULATED", matchIfMissing = true)
public class TerminalSimulatedImpl implements TerminalService { ... }

@ConditionalOnProperty(name = "payment.terminal.type", havingValue = "PHYSICAL")
public class TerminalPhysicalImpl implements TerminalService { ... }
```

@EnableScheduling y @Scheduled

`@EnableScheduling` (en la clase principal) habilita las tareas programadas; `@Scheduled` marca el método que se ejecuta con una periodicidad. Aquí corre `PaymentMonitorJob` cada 5 minutos para revisar pagos `PENDING` antiguos.

```
@Component
public class PaymentMonitorJob {
    @Scheduled(fixedDelay = 300000) // cada 5 min
    public void revisarPagosPendientes() { ... }
}
```

6. Anotaciones de seguridad y permisos

@EnableMethodSecurity

Activa la seguridad a nivel de **método**, es decir, permite usar `@PreAuthorize`. Se coloca una vez en `SecurityConfig`. Sin ella, las anotaciones `@PreAuthorize` se ignoran.

@PreAuthorize

Verifica permisos **antes** de ejecutar el método, usando SpEL. Si no se cumple, Spring lanza `AccessDeniedException` → **403**. Es la pieza central de la autorización del proyecto (34 permisos, uno o varios por endpoint):

```
@PreAuthorize("hasAuthority('CREAR_VENTAS')")
@PreAuthorize("hasAuthority('VER_PRODUCTOS')")
@PreAuthorize("hasRole('ADMIN')") // equivalente a hasAuthority('ROLE_ADMIN')
```

Regla del proyecto: autorizar por PERMISO, no por rol.

`hasAuthority('X')` pregunta por el permiso exacto; `hasRole('ADMIN')` pregunta por el rol (Spring antepone `ROLE_`). Así reasignar permisos a un rol cambia el acceso sin recompilar.

7. Anotaciones de persistencia (JPA/Hibernate)

Estas anotaciones (paquete `jakarta.persistence`) describen cómo se guardan los objetos en PostgreSQL. Hibernate las lee y genera el esquema (`ddl-auto: update`) y las consultas SQL.

@Entity, @Table, @Id, @GeneratedValue, @Column

```
@Entity
@Table(name="roles")
public class RoleEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // id autoincremental
    private Long id;

    @Column(nullable=false, unique=true)
    private String name;
}
```

- **@Entity**: la clase es una tabla; cada instancia, una fila.
- **@Table(name=...)**: nombre explícito de la tabla (si no, se deriva del nombre de clase).
- **@Id**: clave primaria.
- **@GeneratedValue**: cómo se genera el id (`IDENTITY` = columna autoincremental de PostgreSQL).
- **@Column**: restricciones/atributos de la columna (`nullable` , `unique` , `columnDefinition` ...).

@UniqueConstraint

Impone una restricción única compuesta a nivel de tabla (no se puede expresar con `@Column`). En el proyecto evita tener el mismo permiso dos veces en un rol:

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "role_permissions", joinColumns = @JoinColumn(name = "role_id"),
    inverseJoinColumns = @JoinColumn(name="permission_id"),
    uniqueConstraints = {@UniqueConstraint(name = "uk_role_permission",
        columnNames = {"role_id", "permission_id"})})
private Set<PermissionEntity> permissions = new HashSet<>();
```

Relaciones: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany

Anotación	Relación	Ejemplo real
@OneToOne	1 a 1	PaymentEntity → SaleEntity
@ManyToOne	Muchos a 1 (lado dueño, lleva la FK)	SaleDetailEntity → ProductEntity
@OneToMany	1 a muchos (<code>mappedBy</code> , lado inverso)	RoleEntity → UserEntity
@ManyToMany	Muchos a muchos (tabla intermedia)	RoleEntity ↔ PermissionEntity

@JoinColumn nombra la columna de la clave foránea. **@JoinTable** define la tabla intermedia en un N:M (aquí `role_permissions`).

@Enumerated

Guardar un `enum` . `EnumType.STRING` guarda el **nombre** (`APPROVED`) en lugar del ordinal, mucho más seguro ante reordenamientos:

```
@Enumerated(EnumType.STRING)
private PaymentStatus status; // PENDING, APPROVED, REJECTED, REVERSED...
```

Trampa conocida del proyecto

Hibernate crea un CHECK `permissions_name_check` para columnas `@Enumerated(String)`. `ddl-auto: update` **no**

actualiza ese CHECK al agregar valores al enum → si haces eso en una BD existente hay que borrar el constraint (`ALTER TABLE permissions DROP CONSTRAINT permissions_name_check;`) o usar BD nueva.

@PrePersist y @PreUpdate

Callbacks del ciclo de vida: se ejecutan justo antes de insertar (`@PrePersist`) o actualizar (`@PreUpdate`). Se usan para timestamps y valores por defecto:

```
@PrePersist
protected void onCreate(){
    createAt = LocalDateTime.now();
    updateAt = LocalDateTime.now();
    if (attemptCount == null) attemptCount = 0;
}

@PreUpdate
protected void onUpdate(){
    updateAt = LocalDateTime.now();
}
```

8. Transacciones, consultas y fetching

@Transactional

Envuelve el método en una transacción de base de datos: si algo falla, hace **rollback**; si termina bien, **commit**. Es imprescindible cuando una operación toca varias tablas y deben cambiar "todo o nada". Ejemplo real en la creación de una compra (inserta cabecera + renglones, suma stock y actualiza costo):

```
@Transactional
public PurchaseResponse create(PurchaseRequest request){
    // 1) arma cabecera + renglones
    // 2) product.stock += quantity
    // 3) product.cost = unitCost
    // Si algo lanza excepción → rollback de TODO
}
```

Detalle AOP:

`@Transactional` funciona por proxy. Si un método sin la anotación llama internamente a otro método del mismo bean anotado con `@Transactional`, la transacción

no

se crea (la llamada no pasa por el proxy).

@Query

JPQL (o SQL nativo) cuando el nombre del método derivado no alcanza. En el proyecto se usan para agregados de reportes y búsquedas con `LIKE`:

```
@Query("SELECT p FROM ProductEntity p WHERE p.active = true AND " +
        "(LOWER(p.name) LIKE LOWER(CONCAT('%', :q, '%')) OR p.barcode LIKE %:q%)")
List<ProductEntity> search(@Param("q") String q);
```

@EntityGraph

Indica qué relaciones traer en la **misma consulta** (JOIN FETCH), evitando el problema **N+1**: sin él, al recorrer 100 compras se dispararían 100 consultas extra para cargar proveedor y productos. En el proyecto se aplica en los listados de compras:

```
@EntityGraph(attributePaths = {"provider", "details", "details.product"})
List<PurchaseEntity> findAllOrderByDateDesc();
```

9. Manejo de excepciones

@RestControllerAdvice + @ExceptionHandler

`@RestControllerAdvice` marca una clase como "consejera" global: aplica a **todos** los controllers. `@ExceptionHandler` asocia un método a un tipo de excepción; cuando esa excepción se propaga desde cualquier endpoint, Spring llama a ese método en lugar de devolver un 500 genérico. Todo el proyecto centraliza esto en `GlobalExceptionHandler`:

```
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(AccessDeniedException.class) // 403
    public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex) {
        return buildErrorResponse(HttpStatus.FORBIDDEN, "ACCESS_DENIED",
            "No tienes permisos para realizar esta acción");
    }

    @ExceptionHandler(AuthenticationException.class) // 401
    public ResponseEntity<ErrorResponse> handleAuthentication(AuthenticationException ex) {
        return buildErrorResponse(HttpStatus.UNAUTHORIZED, "UNAUTHORIZED", "No autenticado");
    }

    @ExceptionHandler(Exception.class) // 500 genérico
    public ResponseEntity<ErrorResponse> handleGenericException(Exception e) {
        log.error("Error interno del servidor", e); // detalle solo en el servidor
        return buildErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR, "INTERNAL_ERROR",
            "Ocurrió un error interno en el servidor");
    }
}
```

El resultado es un JSON uniforme para el frontend: `{ timestamp, status, code, message }`. También hay handlers para las excepciones de negocio (`ProductException`, `PurchaseException` ...) y para errores de request (`HttpMessageNotReadableException`, `NoResourceFoundException` ...).

Buena práctica aplicada

El mensaje real de una excepción 500

nunca

se expone al cliente: se registra con `log.error` y se responde un texto genérico. Así no se filtran detalles internos (rutas, SQL, stacktrace).

10. Validación de datos (Bean Validation)

Jakarta Bean Validation permite declarar reglas en los DTOs y que Spring las verifique automáticamente cuando se usa `@Valid` en el controller. Si algo falla, se lanza `MethodArgumentNotValidException`, que `GlobalExceptionHandler` convierte en **400** con el detalle por campo.

Anotación	Regla
@NotBlank	No nulo, no vacío y no solo espacios
@NotNull / @NotEmpty	No nulo / colección con al menos un elemento
@Size(min, max)	Longitud o tamaño dentro de rango
@Min / @Max	Valor numérico mínimo / máximo
@Email, @Positive, @Pattern	Formato de email, número positivo, expresión regular

```
// Ejemplo típico (cómo se usarían en un DTO de request):
public class LoginRequest {
    @NotBlank
    @Email
    private String email;

    @NotBlank
    @Size(min = 4)
    private String password;
}

// Y en el controller:
@PostMapping("/login")
public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest r){ ... }
```

Estado real en este proyecto (deuda técnica)

La dependencia `spring-boot-starter-validation` está en el `pom.xml` y el `GlobalExceptionHandler` ya maneja los errores de validación,

pero hoy ningún DTO aplica `@Valid` / `@NotBlank` / `@Min` / `@Size`

: las reglas se validan a mano dentro de los `service/impl` (lanzando `IllegalArgumentException` → 400). Es decir, la infraestructura está lista pero la validación declarativa aún no se adoptó.

11. Serialización JSON (Jackson) y Lombok

@JsonIgnore (Jackson)

Excluye un campo de la serialización/deserialización JSON. En entidades con relaciones bidireccionales evita **ciclos infinitos** (rol → usuarios → rol → ...) y rompe la recursión. En este proyecto se usa en los lados inversos de las relaciones:

```
@JsonIgnore
@OneToMany(mappedBy = "role")
private List<UserEntity> users = new ArrayList<>;
```

Usado en `RoleEntity` (users), `PermissionEntity` y `CategoryEntity` (products).

Lombok

Lombok genera código en tiempo de compilación, así no lo escribes a mano. Este proyecto lo usa **parcialmente**: las entidades antiguas tienen getters/setters manuales, mientras que las nuevas usan Lombok.

Anotación	Qué genera	Ejemplo en el proyecto
@Data	Getters, setters, toString, equals y hashCode	PaymentEntity, TerminalConfig
@Builder	Patrón builder fluido: <code>TestCard.builder().cardNumber(...).build()</code>	TestCard, TerminalRequest/Response
@NoArgsConstructor	Constructor sin argumentos (lo exige JPA)	PaymentEntity
@AllArgsConstructor	Constructor con todos los campos	PaymentEntity
@RequiredArgsConstructor	Constructor con los campos <code>final</code> (inyección por constructor)	mappers/servicios
@Slf4j	Un logger <code>log</code> (SLF4J) listo para usar	GlobalExceptionHandler, TestCardRepository

```
@Slf4j
@Repository
public class TestCardRepository {
    public TestCardRepository(){
        cards.put("4111111111111111", TestCard.builder()
            .cardNumber("4111111111111111")
            .brand("VISA")
            .balance(BigDecimal.valueOf(10000))
            .build());
        log.info("{} tarjetas de prueba cargadas", cards.size());
    }
}
```

@Data y JPA

— Cuidado: `@Data` genera `equals/hashCode` sobre todos los campos, lo que en entidades con relaciones o id autogenerated puede dar problemas. En proyectos grandes se prefiere `@Getter / @Setter + @EqualsAndHashCode(exclude = {id})`.

12. Anotaciones de testing

El proyecto tiene 21 clases de test (130 tests). Se distinguen tres estilos según **cuánto contexto** de Spring cargan y qué sustituyen con mocks.

Anotación	Qué hace	Uso en el proyecto
@SpringBootTest	Levanta todo el contexto de la aplicación	VentasApplicationTests
@DataJpaTest	Solo la capa JPA + BD en memoria (H2)	4 tests de repositorios
@WebMvcTest	Solo la capa web (controllers + MockMvc); el service se mockea	12 tests de controllers
@ExtendWith(MockitoExtension.class)	Test unitario puro con Mockito (sin Spring)	ProductImplTest, PurchaseImplTest, bootstraps
@Mock	Crea un mock (objeto simulado) de una dependencia	tests unitarios
@InjectMocks	Crea la clase bajo test e inyecta los @Mock	PurchaseImplTest
@MockitoBean	Reemplaza un bean del contexto por un mock (Spring Boot)	los 12 @WebMvcTest
@WithMockUser	Simula un usuario autenticado con ciertas authorities	p. ej. <code>@WithMockUser(authorities = "VER_PRODUCTOS")</code>
@Import	Importa una clase (@Configuration) al contexto de test	<code>@Import(SecurityConfig.class)</code>
@BeforeEach	Método que corre antes de cada test (setup)	tests Mockito
@Test	Marca un método como caso de prueba (JUnit 5)	todos
@TempDir	Crea una carpeta temporal que se limpia al terminar	FileStorageServiceTest

```
@WebMvcTest(ProductController.class)
@Import(SecurityConfig.class)
class ProductControllerTest {
    @Autowired private MockMvc mockMvc;           // simula peticiones HTTP
    @MockitoBean private ProductService productService; // service mockeado
    @MockitoBean private JwtUtil jwtUtil;
    @MockitoBean private UserRepository userRepository;
    @MockitoBean private SecurityAuthorityMapper authorityMapper;

    @Test
    @WithMockUser(authorities = "VER_PRODUCTOS")
    void listarProductos_sinFiltro_llamaGetAll() throws Exception {
        when(productService.getAll()).thenReturn(List.of(producto()));
        mockMvc.perform(get("/api/products"))
            .andExpect(status().isOk());
    }
}
```

Por qué se mockean JwtUtil/UserRepository/SecurityAuthorityMapper

`@WebMvcTest` carga `SecurityConfig`, que depende de `JwtFilter` y este de esas clases. Al aportarles mocks, los tests corren sin base de datos ni tokens reales.

13. Tabla maestra: anotación → paquete → uso

Anotación	Paquete	Para qué	Ejemplo en el proyecto
@SpringBootApplication	org.springframework.boot.autoconfigure	Arranque + autoconfiguración + scan	VentasApplication
@Configuration	org.springframework.context.annotation	Clase de beans manuales	SecurityConfig
@Bean	org.springframework.context.annotation	Registrar un objeto	PasswordEncoder, SecurityFilterChain
@Component / @Service / @Repository	org.springframework.stereotype	Estereotipos de bean	JwtUtil / AuthImpl / TestCardRepository
@RestController / @RestControllerAdvice	org.springframework.web.bind.annotation	Controller REST / consejo global JSON	SaleController / GlobalExceptionHandler
@RequestMapping / @GetMapping / @PostMapping / @PutMapping / @DeleteMapping / @PatchMapping	org.springframework.web.bind.annotation	Mapeo de rutas y verbos HTTP	12 controllers
@PathVariable / @RequestParam / @RequestBody	org.springframework.web.bind.annotation	Enlazar URL / query / cuerpo JSON	SaleController
@Autowired	org.springframework.beans.factory.annotation	Inyección de dependencia	tests (@Autowired MockMvc)
@Value	org.springframework.beans.factory.annotation	Injectar una propiedad	JwtUtil (\${JWT_SECRET})
@ConfigurationProperties	org.springframework.boot.context.properties	Enlazar un bloque YAML a un objeto	TerminalConfig
@ConditionalOnProperty	org.springframework.boot.autoconfigure.condition	Bean condicional a una propiedad	TerminalSimulated/PhysicalImpl
@EnableScheduling / @Scheduled	org.springframework.scheduling.annotation	Tareas programadas	PaymentMonitorJob
@Order	org.springframework.core.annotation	Orden de ejecución de beans	los 4 bootstraps
@EnableMethodSecurity / @PreAuthorize	org.springframework.security.config.annotation / ...access.prepost	Autorización por método	SecurityConfig / controllers
@Transactional	org.springframework.transaction.annotation	Transacción (commit/rollback)	PurchaseImpl, SaleImpl, bootstraps
@Entity / @Table / @Id / @GeneratedValue / @Column	jakarta.persistence	Mapeo objeto-relacional	las 12 entidades

@OneToOne / @OneToMany / @ManyToOne / @ManyToMany	jakarta.persistence	Relaciones entre entidades	Payment/Sale, Role/User...
@JoinColumn / @JoinTable / @UniqueConstraint	jakarta.persistence	FK, tabla intermedia, unicidad	RoleEntity (role_permissions)
@Enumerated	jakarta.persistence	Guardar un enum	PaymentEntity (STRING)
@PrePersist / @PreUpdate	jakarta.persistence	Callbacks de ciclo de vida	PaymentEntity (timestamps)
@Query / @EntityGraph	org.springframework.data.jpa.repository	Consultas JPQL / fetch con JOIN	ProductRepository / PurchaseRepository
@ExceptionHandler	org.springframework.web.bind.annotation	Mapear excepción → respuesta	GlobalExceptionHandler
@Valid / @NotBlank / @Min / @Size	jakarta.validation	Validación declarativa (disponible, hoy sin uso)	—
@JsonIgnore	com.fasterxml.jackson.annotation	Excluir campo del JSON	RoleEntity, CategoryEntity
@Data / @Builder / @NoArgsConstructor / @AllArgsConstructor / @RequiredArgsConstructor / @Slf4j	lombok	Generación de código	PaymentEntity, TestCard, GlobalExceptionHandler
@Test / @BeforeEach / @ExtendWith / @TempDir	org.junit.jupiter.api / ...extension	JUnit 5	todos los tests
@SpringBootTest / @DataJpaTest / @WebMvcTest	org.springframework.boot.test / ...autoconfigure	Slices de test	los 21 tests
@Mock / @InjectMocks / @MockitoBean	org.mockito / org.springframework.test.context.bean.override	Mocks y sustitución de beans	tests
@WithMockUser / @Import	org.springframework.security.test / ...context.annotation	Usuario simulado / importar config	tests @WebMvcTest

14. JWT explicado a fondo

14.1 Qué es y por qué se usa

Un **JSON Web Token (JWT)** es una cadena que representa de forma **firmada** un conjunto de datos (los *claims*). En este proyecto, el backend lo entrega tras un login correcto y el frontend lo envía en cada petición para identificarse. Ventajas frente a la sesión clásica:

- **Stateless:** el servidor no guarda sesiones en memoria; toda la identidad va en el token. Escala horizontalmente sin compartir sesión.
- **Verificable sin BD:** la firma permite comprobar que el token lo emitió este servidor y que no fue alterado.
- **Autocontenido:** el payload puede llevar datos (aquí, el email del usuario).

14.2 Estructura: header.payload.signature

El token tiene **tres partes separadas por puntos**, cada una codificada en Base64Url:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJzdWIiOiJhZG1pbk82ZW50YXMuY29tIiwiaWF0IjoxNzU4MDAwMDAwLCJleHAiOjE3NTgwMTgwMDB9
.Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
└── header ─┘      └────────── payload ─────────┘      └──────── signature ─────────┘
```

Parte	Contenido (decodificado)	Función
Header	<code>{"alg":"HS256","typ":"JWT"}</code>	Indica el algoritmo de firma y el tipo
Payload	<code>{"sub":"admin@ventas.com","iat":1758000000,"exp":1758018000}</code>	Los claims (datos) del token
Signature	HMAC-SHA256 de <code>header.payload</code> con el secreto	Garantiza integridad y autenticidad

El payload NO está cifrado

Cualquiera puede decodificar las dos primeras partes con Base64 y leer su contenido. Por eso este proyecto **solo**

mete el email (`sub`) en el token:

nunca

contraseñas, tarjetas ni datos sensibles.

14.3 Base64Url (no es cifrado)

Base64Url es una **codificación**, no un cifrado: convierte bytes en caracteres seguros para URL (usa `-` y `_`, y quita el relleno `=`). Sirve para transportar el JSON. Que sea legible es una decisión de diseño de JWT (autocontenido), no una fuga por sí misma; lo que impide **alterar** el token es la firma.

14.4 Firma HS256 paso a paso

HS256 = HMAC con SHA-256 y un **secreto compartido**. El emisor calcula:

```
firma = HMAC_SHA256(secreto, base64url(header) + "." + base64url(payload))
```

Al receptor le basta recalcular la firma con el mismo secreto y compararla. Si alguien cambia un byte del payload, la firma deja de coincidir → token inválido. Por eso **el secreto es lo más crítico**: quien lo tenga puede firmar tokens válidos (incluido uno de ADMIN).

```
// JwtUtil.java - el secreto llega de ${JWT_SECRET} (≥ 32 bytes)
@Value("${JWT_SECRET}")
private String secretKey;

.signWith(Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8)),
    SignatureAlgorithm.HS256)
```

14.5 Claims y expiración

Claim	Significado	Valor en este proyecto
sub	Subject: de quién es el token	email del usuario
iat	Issued At: cuándo se emitió	fecha de login
exp	Expiration: cuándo caduca	emisión + 5 horas

```
// 5 horas = 1000 ms · 60 s · 60 min · 5
.setExpiration(new Date(System.currentTimeMillis() + 1000L * 60 * 60 * 5))
```

Al caducar, `parseClaimsJws` lanza `ExpiredJwtException`, el filtro no autentica y el endpoint responde **401**. El usuario debe volver a iniciar sesión.

14.6 Flujo completo en este proyecto

Fase 1 — Login (obtener el token)

```
Cliente
| POST /api/auth/login { email, password }
▼
AuthController → AuthImpl.login()
| userRepository.findByEmailWithRoleAndPermissions(email)
| · filtro isActive()
| · passwordEncoder.matches(password, user.password) // BCrypt
| · jwtUtil.generateToken(user) // firma HS256
▼
200 OK { success:true, token:"eyJ...", rol, permisos[...] }
└ el frontend guarda el token y lo adjunta en cada llamada
```

Fase 2 — Petición autenticada (validar el token)

```
Cliente
| GET /api/sales Authorization: Bearer eyJ...
▼
SecurityFilterChain
▼
JwtFilter (OncePerRequestFilter, antes de UsernamePasswordAuthenticationFilter)
1. ¿Hay header "Bearer "? si no → sigue sin autenticar
2. jwtUtil.extractEmail(token) → email (subject)
3. jwtUtil.isTokenValid(token) → ¿firma OK y no expirado?
4. userRepository.findByEmailWithRoleAndPermissions(email)
5. SecurityAuthorityMapper.mapAuthorities(user)
   → [ROLE_ADMIN, VER_VENTAS, CREAR_VENTAS, ...]
6. SecurityContextHolder.setAuthentication(...)
▼
Continuar cadena → Controller → @PreAuthorize("hasAuthority('VER_VENTAS')")
├ permiso presente → ejecuta el método → 200
└ permiso ausente → AccessDeniedException → 403
```

14.7 Decisión clave: los permisos se leen de la BD, no del token

El JWT del proyecto solo lleva el **email**. En cada request, `JwtFilter` vuelve a cargar el usuario con su rol y permisos desde la base de datos:

```
UserEntity user = userRepository.findByEmailWithRoleAndPermissions(email).orElse(null);
UsernamePasswordAuthenticationToken authToken =
    new UsernamePasswordAuthenticationToken(user, null, authorityMapper.mapAuthorities(user));
SecurityContextHolder.getContext().setAuthentication(authToken);
```

Ventaja

Si a un rol se le quita un permiso, el cambio aplica

de inmediato

, sin esperar a que expire el token. Si los permisos viajaran dentro del JWT, un permiso revocado seguiría siendo válido hasta la expiración (5 h). El precio es una consulta por request (se mitiga con `JOIN FETCH`).

14.8 Piezas de la implementación

Clase	Rol
JwtUtil	Genera el token (generateToken) y lo valida/parsea (isTokenValid, extractEmail)
JwtFilter	Extrae el token del header, lo valida, carga el usuario y llena el SecurityContext
SecurityAuthorityMapper	Convierte rol + permisos en autoridades (<code>ROLE_*</code> y cada permiso)
SecurityConfig	Rutas públicas, CORS, CSRF off, registro de JwtFilter, <code>@EnableMethodSecurity</code>
AuthImpl	Login, forgot/reset/change password

```
// SecurityAuthorityMapper: ROLE_ + un authority por permiso
authorities.add(new SimpleGrantedAuthority("ROLE_" + user.getRole().getName()));
for (PermissionEntity permission : user.getRole().getPermissions()) {
    authorities.add(new SimpleGrantedAuthority(permission.getName().name()));
}
```

```
// SecurityConfig: rutas públicas y filtro JWT
.csrf(csrf -> csrf.disable())
.cors(Customizer.withDefaults())
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/api/uploads/**").permitAll()
    .anyRequest().authenticated())
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
```

14.9 Manejo de token inválido o expirado

El filtro **nunca** lanza un 500 por un token malo: captura `JwtException` (firma errónea, expirado, malformado) e `IllegalArgumentException`, limpia el contexto y deja seguir la cadena. El endpoint protegido responderá **401** (sin autenticar) o **403** (autenticado pero sin permiso).

```
} catch (JwtException | IllegalArgumentException e) {
    SecurityContextHolder.clearContext();
    filterChain.doFilter(request, response); // sin autenticación → 401
    return;
}
```

14.10 Seguridad: buenas prácticas y riesgos

Práctica	En este proyecto
Secreto fuerte (≥ 32 bytes) y fuera del código	<code>\$_JWT_SECRET</code> en env vars / <code>application-local.yaml</code> (no versionado)
No meter datos sensibles en el payload	Solo el email (sub)
HTTPS en producción	Dominio *.up.railway.app con HTTPS automático
Expiración razonable	5 horas
Contraseñas con hash	BCrypt (PasswordEncoder)
CSRF	Desactivado porque la API es stateless y no usa cookies de sesión
CORS restringido	<code>\$_CORS_ALLOWED_ORIGINS</code> (localhost en dev, Netlify en prod)

Riesgos a tener en cuenta

- **Robo del token:**
quien lo intercepte puede usarlo hasta que expire. Mitigación: HTTPS, expiración corta, no guardarlo de forma insegura.
- **XSS vs localStorage:**
si el frontend guarda el token en `localStorage`, un XSS puede leerlo. Alternativa: cookie `HttpOnly` + `SameSite`.
- **Revocación:**
un JWT puro no se puede "anular" antes de expirar. Aquí se compensa releyendo permisos de la BD en cada request; para invalidar el login entero haría falta una lista negra o un refresh token.
- **Refresh tokens:**
no implementados; al expirar (5 h) el usuario vuelve a iniciar sesión.
- **Secreto filtrado:**
si el secreto acaba en Git, cualquiera puede forjar un token de ADMIN. Por eso hay que purgar el historial antes de publicar el repo.

14.11 JWT vs sesión clásica

Aspecto	Sesión con cookie	JWT (este proyecto)
Estado en el servidor	Sí (memoria/BD)	No (stateless)
Escalado horizontal	Requiere sesión compartida	Directo
Revocación inmediata	Fácil (borrar sesión)	Difícil (hasta expirar)
Transporte	Cookie automática	Header Authorization: Bearer
CSRF	Riesgo (cookie)	No aplica igual (no cookie)

15. Preguntas de entrevista

Sobre anotaciones

1. **¿Diferencia entre @Component, @Service y @Repository?** Los tres crean un bean; @Service es semántico y @Repository añade traducción de excepciones de persistencia. La funcionalidad base es la misma.
2. **¿@Bean vs @Component?** @Component en la clase (autoescaneo); @Bean en un método de una @Configuration, útil para clases de terceros que no puedes anotar.
3. **¿Cómo funciona @Autowired?** Spring resuelve la dependencia por tipo y la inyecta (constructor, setter o campo). Con un solo constructor es implícito.
4. **¿Por qué es mejor la inyección por constructor?** Campos final, detección temprana de ciclos, testeo más simple sin reflexión.
5. **¿Qué hace @Transactional y por qué puede no funcionar?** Abre una transacción alrededor del método por proxy AOP; no funciona en llamadas internas del mismo bean ni en métodos privados.
6. **¿Para qué sirve @EntityGraph?** Traer relaciones con JOIN en la misma consulta para evitar el problema N+1.
7. **¿@Enumerated(STRING) vs ORDINAL?** STRING guarda el nombre (seguro ante reordenamientos); ORDINAL guarda el índice (frágil).
8. **¿Qué resuelve @RestControllerAdvice + @ExceptionHandler?** Centralizar el manejo de errores y unificar el formato de respuesta.
9. **¿@PreAuthorize en qué momento se evalúa?** Antes de invocar el método, vía proxy; si falla lanza AccessDeniedException (403).
10. **¿Qué hace @ConditionalOnProperty?** Registrar un bean solo si una propiedad tiene un valor concreto (aquí, la terminal SIMULATED o PHYSICAL).

Sobre JWT

1. **¿Qué es un JWT y cuántas partes tiene?** Token firmado con tres partes Base64Url: header, payload y signature.
2. **¿El payload está cifrado?** No: está codificado y es legible. Nunca se meten datos sensibles.
3. **¿Cómo se verifica la autenticidad?** Recalculando la firma HMAC-SHA256 con el secreto y comparándola.
4. **¿Qué pasa si alguien cambia el payload?** La firma deja de coincidir y el token se rechaza.
5. **¿Dónde se guarda el token en el cliente?** En memoria o localStorage (con riesgo de XSS); alternativa más segura: cookie HttpOnly.
6. **¿Cómo se revoca un JWT?** Un JWT puro no se revoca; hay que usar expiración corta, lista negra o refresh tokens. Este proyecto relee los permisos de la BD en cada request.
7. **¿Qué ventaja tiene leer los permisos de la BD y no del token?** Los cambios de permisos aplican al instante, sin esperar a la expiración.
8. **¿Por qué se desactiva CSRF con JWT?** Porque no se usan cookies de sesión; el token viaja en un header que un sitio externo no puede leer ni enviar automáticamente.
9. **¿Qué pasa cuando el token expira?** JwtUtil lanza ExpiredJwtException, el filtro no autentica y el endpoint devuelve 401; el usuario debe loguearse de nuevo.
10. **¿Qué longitud debe tener el secreto HS256?** Al menos 256 bits (32 bytes); el proyecto lo exige vía jjwt (Keys.hmacShaKeyFor).

16. Glosario

Anotación	Metadato en Java que Spring lee para configurar el comportamiento.
Bean	Objeto administrado por el contenedor de Spring.
ApplicationContext	El contenedor: guarda y conecta todos los beans.
Classpath scanning	Búsqueda de clases anotadas para registrarlas como beans.
Inyección de dependencias (DI)	Spring entrega a cada bean las dependencias que necesita.
AOP / proxy	Programación orientada a aspectos; Spring envuelve beans para añadir lógica (transacciones, seguridad).
Estereotipo	Anotación que marca el rol de una clase (@Service, @Repository...).
JPA / Hibernate	Especificación de persistencia / su implementación.
N+1	Problema de rendimiento: 1 consulta + N extra al recorrer relaciones lazy.
JPQL	Lenguaje de consultas de JPA, orientado a entidades.
Bean Validation	Validación declarativa con @NotBlank, @Size, @Min...
Slice test	Test que carga solo una capa (@WebMvcTest, @DataJpaTest).
Mock	Objeto simulado que sustituye a una dependencia real en un test.
JWT	JSON Web Token: cadena firmada y autocontenida con claims.
Claim	Un dato dentro del payload (sub, iat, exp...).
sub / iat / exp	Subject (identidad), emitido en, expira en.
HS256	HMAC con SHA-256 y secreto compartido.
Base64Url	Codificación segura para URL; no es cifrado.
Bearer token	Token enviado en <code>Authorization: Bearer <token></code> .
Stateless	Sin estado de sesión en el servidor.
CSRF	Cross-Site Request Forgery: ataque que abusa de cookies de sesión.
CORS	Reglas de qué orígenes pueden llamar a la API desde el navegador.
BCrypt	Algoritmo de hash de contraseñas con salt incorporado.
Refresh token	Token de larga duración para obtener nuevos access tokens sin re-loguear.

Documento generado para el proyecto **Ventas-Backend** (Spring Boot 4 · Java 21). Complementa la *Guía de Spring Boot*. Todas las anotaciones listadas existen en el código actual del repositorio.